

[illegible]

String: Sequence of Data

A string literal in Java is basically a sequence of characters. These characters can be anything like alphabets, numbers, special characters which are enclosed with double quotes. So we can say String is a collection of alphanumeric characters.

Internally in java, it is a character array.

Example : String str = "May";



How can you create String in Java :-

In java, String can be created by using 3 ways :-

- 1) By using String Literal
`String s = "Ravi";`
- 2) By using new keyword
`String s = new String("RaviRavi");`
- 3) By using character array (old technique)
`char ch[] = {'R','V','I','R','V','I'};`

- What is immutability in String?
- It means that the value of a String object cannot be modified. This means that the String object either created by using the String constructor or by using the String literal are immutable.
- If we want to perform any operation then the operation will be performed in a new memory location.
- Whenever we create a String object using the String constructor, the String object is automatically placed in a special area of the JVM memory called "String Constant Pool".
- Actually, internally JVM is calling a method called `intern()` to place the String object inside the String constant pool area.

Case 2:

```
String c1 = "1000";  
c1.toUpperCase();  
System.out.println(c1);
```

The diagram illustrates the state of memory for the variable `c1`. A line labeled `c1` points to a cloud-shaped memory block. Inside the cloud, the value `1000` is written in black, and the word `HEAP` is written in red below it. At the bottom of the cloud, there is a small icon of a trash can with the letters `GC` next to it, representing the garbage collection mechanism.

Case 3:

```
String s1 = "test1";
s1 = s1.toLowerCase();
System.out.println(s1);
```

The diagram shows a client with IP 192.168.1.100 connected to a cloud service. The cloud contains a server with IP 192.168.1.101. The cloud is labeled 'cloud' and 'server'.

Case 4 :

```
String s2 = "India";  
String s2 = s1.toUpperCase();  
((String)s2).compareTo(s1)  
((String)s2).compareTo(s1.toUpperCase())
```

The diagram illustrates the memory state for Case 4. On the left, a vertical rectangle represents the memory stack. Two pointers, labeled 's1' and 's2', are shown. Both pointers have arrows pointing to a single memory cell. This memory cell contains the text 'INDIA' in all capital letters. Above the 'INDIA' text, the number '2000' is written, indicating the memory address. The entire memory cell is enclosed in a cloud-like shape, and the label 'REF' is placed below it.

Facts about String and memory:

- String s1 = "Java";
- String s2 = "Java";
- String s3 = "Java";

The diagram illustrates that all three String variables (s1, s2, s3) point to the same memory location. A box labeled 'Java' is shown with a memory address '1800x' next to it. Three arrows, labeled s1, s2, and s3, all point to this box. Below the box, the text '507' is visible.

It just whenever we create a new String object by using String.valueOf(), first of all JVM will verify whether the String is available in pre-existing (already available) in the String constant pool or not.

If the String is pre-existing (already available) in the String Constant pool then JVM will not create any new String object, the same old existing String object would be refer by new reference variable as shown in the diagram.

Note :- In J2SE area we can't have duplicate String Objects.